

Documento de Requerimientos de Producto (PRD)

# Sistema de Análisis Estadístico para Básquet Formativo

Estado: **Borrador Técnico** | Versión: 1.0

Ingeniería de Software · Proyecto Estadísticas Básquet

31 de mayo de 2026

|                          |                                                            |
|--------------------------|------------------------------------------------------------|
| <b>ID del documento:</b> | PRD-BSKT-2026-001                                          |
| <b>Producto:</b>         | StatsPro Basketball (Nombre en clave)                      |
| <b>Ingeniería:</b>       | Equipo de 2 Ingenieros de Software                         |
| <b>Clasificación:</b>    | Interno — Desarrollo Profesional                           |
| <b>Stack Principal:</b>  | SQLite (Local), Python/Pandas (Análisis), Java/Python (UI) |

Índice

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>1. Descripción General del Producto</b>                          | <b>3</b>  |
| 1.1. Planteamiento del Problema . . . . .                           | 3         |
| 1.2. Visión del Producto . . . . .                                  | 3         |
| 1.3. Métricas de Éxito . . . . .                                    | 3         |
| <b>2. Perfiles de Usuario</b>                                       | <b>4</b>  |
| <b>3. Metas y No Metas</b>                                          | <b>4</b>  |
| 3.1. En el Alcance (v1.0) . . . . .                                 | 4         |
| 3.2. Fuera de Alcance . . . . .                                     | 4         |
| <b>4. Acuerdo de Ingeniería y Estándares</b>                        | <b>4</b>  |
| 4.1. Principios de Desarrollo . . . . .                             | 4         |
| 4.2. Calidad de Código y Pruebas . . . . .                          | 5         |
| <b>5. Gestión de Tareas</b>                                         | <b>5</b>  |
| <b>6. Arquitectura de Datos (Basado en DER)</b>                     | <b>5</b>  |
| 6.1. Entidades Principales . . . . .                                | 5         |
| 6.2. Reglas de Integridad . . . . .                                 | 5         |
| <b>7. Arquitectura de Software (Clean + Hexagonal)</b>              | <b>5</b>  |
| 7.1. Capas y Responsabilidades . . . . .                            | 5         |
| 7.2. Regla de Dependencias . . . . .                                | 6         |
| 7.3. Patrones de Diseño . . . . .                                   | 6         |
| 7.4. Estructura Objetivo de Directorios . . . . .                   | 6         |
| <b>8. Registro de Decisiones Arquitectónicas (ADR)</b>              | <b>6</b>  |
| <b>9. Gestión de Tareas y Plan de Entregas (Consolidado)</b>        | <b>7</b>  |
| 9.1. Hito 1: Núcleo de Datos e Interfaz CLI (v0.1) . . . . .        | 7         |
| 9.2. Hito 2: Motor de Ingesta y Análisis (v0.2) . . . . .           | 11        |
| 9.3. Hito 3: Visualización Pro y Reporting (v0.3) . . . . .         | 13        |
| 9.4. Hito 4: Interfaz Multiplataforma y Entrega (v1.0) . . . . .    | 14        |
| <b>10.Reglas de Negocio Consolidadas</b>                            | <b>15</b> |
| <b>11.Requisitos No Funcionales (NFR)</b>                           | <b>16</b> |
| <b>12.Definición de "Hecho"(Definition of Done)</b>                 | <b>16</b> |
| 12.1. Catálogo técnico de criticidad . . . . .                      | 17        |
| <b>13.Hoja de Ruta y Futuras Expansiones</b>                        | <b>17</b> |
| 13.1. Hito 5: Ecosistema Conectado y Colaborativo (v1.5) . . . . .  | 17        |
| 13.2. Hito 6: Inteligencia Táctica Basada en IA (v2.0) . . . . .    | 18        |
| 13.3. Hito 7: Live Stats y Scouting en Tiempo Real (v3.0) . . . . . | 18        |

## 1 Descripción General del Producto

### 1.1 Planteamiento del Problema

El análisis estadístico en el básquet formativo sufre de una brecha crítica entre la recolección de datos y la utilidad táctica. Los entrenadores operan bajo condiciones de alta presión donde la toma de decisiones basada en datos es limitada por herramientas fragmentadas.

Tabla 1: Matriz de Problemas Operativos

| # | Problema                  | Impacto Operativo                                                     | Línea Base                     |
|---|---------------------------|-----------------------------------------------------------------------|--------------------------------|
| 1 | Carga manual ineficiente. | Interferencia táctica durante el juego; pérdida de foco.              | >20 min/partido.               |
| 2 | Ceguera estadística.      | Decisiones basadas en intuición, no en métricas avanzadas (PPP, Eff). | Análisis 0 % automatizado.     |
| 3 | Fragmentación de datos.   | Imposibilidad de seguimiento histórico o comparativo de rivales.      | Datos aislados en papel/Excel. |
| 4 | Inestabilidad de red.     | Apps web fallan en estadios sin conectividad.                         | Requiere 100 % online.         |
| 5 | Rigidez de plataformas.   | El DT no puede integrar los datos de "Ges Deportivo" fácilmente.      | Ingesta manual post-partido.   |

### 1.2 Visión del Producto

Desarrollar una aplicación multiplataforma (PC y Mobile) de ejecución local que centralice el conocimiento deportivo en un motor estadístico profesional. El sistema permitirá una gestión integral **sin necesidad de internet**, con un flujo de trabajo optimizado mediante la carga de planillas de "Ges Deportivo".

### 1.3 Métricas de Éxito

- **MTTR (Time to Report):** Tiempo desde la carga del archivo hasta el informe completo <2 minutos.
- **Tasa de Automatización:** % de estadísticas de partido generadas sin intervención manual >95 %.
- **Disponibilidad Offline:** 100 % de las funcionalidades críticas operativas sin conexión.
- **North Star Metric:** Número de sesiones de análisis táctico realizadas por el DT por semana.

## 2 Perfiles de Usuario

| Perfil               | Rol y Contexto                                        | Objetivo Principal                                            | Problema Actual                                        |
|----------------------|-------------------------------------------------------|---------------------------------------------------------------|--------------------------------------------------------|
| Director Técnico     | Líder táctico en categorías formativas a profesional. | Optimizar el rendimiento del equipo mediante datos objetivos. | El cálculo manual de Eff/PPP es tedioso.               |
| Analista / Asistente | Responsable de la carga y procesamiento de datos.     | Proveer informes rápidos y precisos al DT.                    | Carga redundante de datos existentes en Ges Deportivo. |

## 3 Metas y No Metas

### 3.1 En el Alcance (v1.0)

- **Ingesta Multimodal:**
  - **Automática:** Carga de planillas Excel de "Ges Deportivo".
  - **Manual:** Formulario para carga post-partido (jugador por jugador).
- **Persistencia Local:** Base de datos SQLite única por instalación de usuario.
- **Motor Estadístico:** Análisis de tendencias, PPP, porcentajes por zona y eficiencia con Pandas.
- **Multiplataforma:** Ejecución en Windows, Linux, Mac y dispositivos móviles.

### 3.2 Fuera de Alcance

- **Carga en Vivo (v1.0):** La funcionalidad de toma de datos en tiempo real se posterga para versiones post-estables.
- **Sincronización Cloud:** No se contempla almacenamiento en la nube inicialmente.
- **Importación Automática/API:** No hay conexión directa con servidores de Ges Deportivo (flujo manual).

## 4 Acuerdo de Ingeniería y Estándares

El equipo se compromete a seguir los siguientes principios para garantizar un software profesional y mantenible:

### 4.1 Principios de Desarrollo

- **Diseño Precede a la Implementación:** No se escribe código sin un diseño previo aprobado (ADR).
- **Atomicidad:** Confirmaciones (commits) pequeñas y lógicas. Formato: `tipo(alcance): descripción`.
- **Gestión de Ramas:** Uso de `feature/nombre-tarea`, `hotfix/descripción` y `release/vX.Y`.

- **Higiene del Repositorio:** Prohibido subir binarios, bases de datos SQLite con datos reales o archivos temporales.

## 4.2 Calidad de Código y Pruebas

- **Documentación:** Estilo Javadoc/Doxygen para todos los módulos y métodos públicos.
- **Pruebas:** Cobertura mínima del 80 % en la lógica de negocio (especialmente en el motor estadístico).
- **Análisis Estático:** Uso de `flake8`/`pylint` para Python y `Checkstyle` para Java.

## 5 Gestión de Tareas

---

- **Prioridad:** Urgente (bloqueante), Alta (impacto directo), Media (importante), Baja (mejora).
- **Tamaño (Esfuerzo):** XS (<1 día), S (1-2 días), M (3-5 días), L (6-10 días).

## 6 Arquitectura de Datos (Basado en DER)

---

El sistema utiliza una base de datos relacional local (**SQLite**) con el siguiente esquema normalizado:

### 6.1 Entidades Principales

- **Gestión de Identidad:** `usuario`, `club`, `usuarioClub` (N:M).
- **Estructura Deportiva:** `jugador`, `categoría`, `competencia`, `inscripcion`.
- **Gestión de Listas:** `listaBuenaFe`, `jugadorListaBuenaFe`.
- **Eventos y Estadísticas:** `partido`, `jugadorPartido` (Contiene 20+ métricas: Puntos, T2, T3, TL, Rebotes, Asistencias, etc.).

### 6.2 Reglas de Integridad

- **Vínculos:** Relación 1:1 entre `inscripcion` y `listaBuenaFe`.
- **Persistencia:** El historial de un jugador en diferentes clubes se mantiene vía `jugadorClub`.
- **Métricas:** La tabla `jugadorPartido` actúa como el hecho (fact table) para el motor de análisis.

## 7 Arquitectura de Software (Clean + Hexagonal)

---

Para cumplir portabilidad, testabilidad y ejecución local-first, el sistema adopta una Arquitectura Limpia con enfoque Hexagonal.

### 7.1 Capas y Responsabilidades

- **Dominio (src/domain):** Entidades puras, reglas de negocio, excepciones e interfaces (ports) sin dependencias externas.
- **Aplicación (src/application):** Casos de uso y DTOs; orquesta reglas usando inyección de dependencias.

- **Infraestructura (src/infrastructure):** Repositorios SQLite, parser Excel (Pandas), UI CLI/GUI y generación de reportes.

## 7.2 Regla de Dependencias

- `domain <- application <- infrastructure`
- `domain` no puede importar `application` ni `infrastructure`.
- `application` puede importar `domain`, pero no `infrastructure`.

## 7.3 Patrones de Diseño

- **Repository Pattern:** Abstracción de persistencia por agregado (`UserRepository`, `PlayerRepository`, etc.).
- **Dependency Injection:** Los casos de uso reciben puertos, no implementaciones concretas.
- **Command Pattern:** CLI extensible por subcomandos sin bloques monolíticos `if/else`.

## 7.4 Estructura Objetivo de Directorios

- `src/domain/entities`, `interfaces`, `exceptions`
- `src/application/use_cases`, `dtos`, `services`
- `src/infrastructure/persistence`, `analytics`, `ingest`, `reports`, `ui`
- `tests/unit`, `tests/integration`

## 8 Registro de Decisiones Arquitectónicas (ADR)

Las siguientes decisiones son bloqueantes para hitos sucesivos:

| ID      | Título                   | Estado    | Decisión                                                                   | Bloquea |
|---------|--------------------------|-----------|----------------------------------------------------------------------------|---------|
| ADR-001 | Arquitectura Local-First | Aprobado  | SQLite + operación offline-first.                                          | Hito 1  |
| ADR-002 | Framework UI             | Pendiente | Selección entre Flet y Compose Multiplatform.                              | Hito 2  |
| ADR-003 | Protocolo de Ingesta     | Pendiente | Estandarizar mapeo y limpieza de Excel Ges Deportivo.                      | US-201  |
| ADR-004 | Versionado de DB         | Pendiente | Definir migraciones ( <code>schema_version</code> o herramienta dedicada). | Hito 2  |
| ADR-005 | Reportes PDF             | Pendiente | Seleccionar librería PDF local (ej. ReportLab).                            | US-302  |
| ADR-006 | Seguridad y Cifrado      | Pendiente | Hash de contraseñas y eventual cifrado de DB (SQLite).                     | Hito 4  |

|         |                        |           |                                                     |        |
|---------|------------------------|-----------|-----------------------------------------------------|--------|
| ADR-007 | Motor de Visualización | Pendiente | Selección de librería de gráficos para dashboards.  | US-301 |
| ADR-008 | Estrategia de Backup   | Pendiente | Exportación/restauración de base local y versiones. | Hito 4 |

## 9 Gestión de Tareas y Plan de Entregas (Consolidado)

*Nota operativa:* los checklist de criterios de aceptación en este PRD son plantillas para crear issues/tareas. El estado real ([ ]/[x]) se gestiona en el tablero de trabajo y no en este archivo versionado.

### 9.1 Hito 1: Núcleo de Datos e Interfaz CLI (v0.1)

**Objetivo del hito:** construir una base técnica funcional por CLI, con persistencia robusta, autenticación local, casos de uso operativos y validaciones de integridad.

#### Épica E1 — Infraestructura y Persistencia

##### ■ US-101: Implementación de Base de Datos SQLite y Vistas

- *Objetivo funcional:* habilitar un esquema relacional local verificable para todo el ciclo del producto.
- *Capa de infraestructura:* clase `SQLiteManager` en `src/infrastructure/persistence/database_manager` con métodos de conexión e inicialización atómica.
- *Base SQL obligatoria:* `src/infrastructure/persistence/sql/schema.sql`, `views.sql`, `seed.sql`.
- *Detalle de vistas a desarrollar:*
  - `v_partidos_resumen`: una fila por partido con competencia, fecha, club local/visitante (nombre), marcador final y estado.
  - `v_boxscore_completo`: una fila por jugador-partido con identificación de partido, club, jugador, minutos y rubros estadísticos crudos.
  - `v_jugador_totales_temporada`: acumulados y promedios por jugador-temporada-competencia para ranking y líderes.
  - `v_listas_detalle`: jugadores habilitados por inscripción/lista de buena fe con estado de habilitación y club asociado.
- *Criterios de aceptación técnicos:*
  - [ ] AC1: schema idempotente con `CREATE TABLE IF NOT EXISTS`.
  - [ ] AC2: FKs activas y reglas `ON DELETE/UPDATE CASCADE` definidas en relaciones críticas.
  - [ ] AC3: `CHECK constraints` aplicadas en métricas y campos de integridad.
  - [ ] AC4: vistas operativas `v_partidos_resumen`, `v_boxscore_completo`, `v_jugador_totales_temporada`, `v_listas_detalle`.
  - [ ] AC5: todas las divisiones de vistas protegidas con `NULLIF/CASE`.
  - [ ] AC6: datos semilla mínimos de referencia: 1 usuario, 2 clubes, 10 jugadores (5 por club), 1 competencia y 3 partidos con boxscore completo.

- [ ] AC7: definición de columnas estable (nombres y tipos) para consumo desde Pandas sin transformaciones ambiguas.
- *Reglas de negocio DB*: `idClubLocal != idClubVisitante`, coherencia `fechaHasta >= fechaDesde`, unicidad en `listaBuenaFe.idInscripcion`.
- *Testing mínimo*:
  - Unitario SQL smoke-test: ejecución limpia de `schema.sql`, `views.sql`, `seed.sql`.
  - Integración: `tests/integration/test_database_schema.py` para existencia de tablas/vistas, FKs y constraints.
  - Integración por vista: asserts de columnas esperadas y de semántica de datos para las 4 vistas.
  - Integración de robustez: casos de división por cero en vistas deben devolver 0.0 o NULL controlado, nunca error.
- **US-102: DatabaseManager y Patrón Repository**
  - *Objetivo funcional*: separar acceso a datos por agregados de dominio bajo contratos tipados.
  - *Dominio (ports)*: `src/domain/interfaces/user_repository.py`, `club_repository.py`, `player_repository.py`, `game_repository.py`.
  - *Infraestructura (implementaciones)*: `src/infrastructure/persistence/sqlite_user_repository.py`, `sqlite_club_repository.py`, `sqlite_player_repository.py`, `sqlite_game_repository.py`.
  - *Criterios de aceptación técnicos*:
    - [ ] AC1: `connect()` activa FKs y `row_factory=sqlite3.Row`.
    - [ ] AC2: los repositorios retornan dataclasses, no tuplas SQL.
    - [ ] AC3: la capa `domain` no importa `sqlite3/pandas`.
    - [ ] AC4: guardado de partido + boxscore con transacción atómica.
  - *Testing mínimo*: `tests/integration/test_repositories.py` con CRUD, búsquedas, mapeos y atomicidad.

## Épica E2 — Lógica de Aplicación y CLI

- **US-103: Gestión de Entidades (Casos de Uso)**
  - *Objetivo funcional*: implementar lógica de negocio para jugadores, clubes, competencias e inscripciones.
  - *Entidades a crear*: `src/domain/entities/usuario.py`, `club.py`, `jugador.py`, `jugador_club.py`, `competencia.py`, `inscripcion.py`, `partido.py`, `estadistica_jugador.py`.
  - *Use cases a crear*: `src/application/use_cases/registrar_jugador.py`, `crear_club.py`, `vincular_jugador_club.py`, `crear_competencia.py`, `inscribir_club_competencia.py`, `listar_clubes_usuario.py`, `listar_jugadores_club.py`, `listar_partidos_por_club.py`.
  - *DTOs frontera*: `src/application/dtos/jugador_dto.py`, `club_dto.py`, `competencia_dto.py`, `inscripcion_dto.py`, `partido_resumen_dto.py`.
  - *Criterios de aceptación técnicos*:
    - [ ] AC1: validación fail-fast de DNI inválido o duplicado.



- [ ] AC2: control de afiliaciones activas para evitar superposición de vínculos.
- [ ] AC3: alta de entidades con reglas de dominio verificadas antes de persistir.
- [ ] AC4: operaciones de inscripción con comportamiento atómico.
- [ ] AC5: listados CLI tabulados y consistentes con vistas SQL.
- *Testing mínimo:* `tests/unit/test_entities.py`, `tests/unit/test_use_cases_admin.py`, más integración en SQLite `:memory:`.

#### ■ US-104: Autenticación y Sesión Local

- *Objetivo funcional:* registro/login seguro y persistencia de sesión entre ejecuciones.
- *Archivos a crear:* `src/infrastructure/security/password_hasher.py`, `src/application/services/`  
`src/application/use_cases/registrar_entrenador.py`, `src/application/use_cases/login_local.py`,  
`src/application/dtos/auth_dto.py`.
- *Comandos CLI:* `stats auth register`, `stats auth login`, `stats auth logout`.
- *Criterios de aceptación técnicos:*
  - AC1: contraseñas nunca en texto plano.
  - AC2: sesión persistente en archivo local seguro.
  - AC3: soporte de `club_activo_id` en contexto de sesión.
  - AC4: validación de email único y políticas de password.
- *Testing mínimo:* `tests/unit/test_auth.py` + pruebas de sesión con archivo temporal.

#### ■ US-105: CargarPartido con Transacción Atómica

- *Objetivo funcional:* persistir partido y estadísticas en operación todo-o-nada.
- *Archivos a crear:* `src/application/dtos/partido_dto.py`, `src/application/use_cases/cargar_partido.py`,  
`tests/unit/test_use_case_cargar_partido.py`.
- *Criterios de aceptación técnicos:*
  - [ ] AC1: rollback completo ante cualquier error parcial del boxscore.
  - [ ] AC2: validación completa de DTOs antes de la primera operación de DB.
  - [ ] AC3: mensajes de error con jugador/campo inválido identificable.
  - [ ] AC4: casos de uso sin SQL embebido (delegación total al repositorio).
- *Reglas críticas:* `convertidos <= lanzados`, minutos válidos, no negativos y consistencia de puntos por fórmula.
- *Testing mínimo:* unit + integración con DB en memoria verificando rollback.

#### ■ US-106: CLI con Command Pattern

- *Objetivo funcional:* CLI extensible y mantenible con subcomandos desacoplados.
- *Archivos a crear:* `src/infrastructure/ui/cli/main_cli.py`, `src/infrastructure/ui/cli/commands/`  
`club_commands.py`, `player_commands.py`, `game_commands.py`, `src/infrastructure/ui/cli/formatters/`  
`club_formatter.py`, `player_formatter.py`, `game_formatter.py`.
- *Criterios de aceptación técnicos:*
  - [ ] AC1: comandos protegidos requieren sesión autenticada cuando corresponda.
  - [ ] AC2: comandos de dominio requieren club activo cuando aplique.

- [ ] AC3: listados de lectura consumen vistas SQL y no consultas ad-hoc dispersas.
  - [ ] AC4: errores funcionales se muestran sin traceback técnico al usuario final.
- *Testing mínimo:* tests de flujo de comandos, permisos y formato de salida.
- **US-107 (Nueva): Monitoreo y Trazabilidad Operativa**
  - *Objetivo funcional:* observabilidad transversal para depuración y soporte.
  - Archivos a crear: `src/infrastructure/logging/logger_config.py`, `src/application/services/execution_context.py`, `src/infrastructure/logging/seq_handler.py`, `docker-compose.yml` (perfil opcional observabilidad con Seq), `tests/unit/test_execution_context.py`, `tests/unit/test_logging_redaction.py`.
  - *Eventos de log obligatorios:*
    - inicio/fin de comando CLI, con resultado (OK/ERROR) y duración;
    - login, logout, cambio de club activo y fallos de autenticación;
    - inicio/fin de transacciones críticas (carga partido, importación Excel, backup/restore);
    - creación/actualización de entidades principales (club, jugador, competencia, partido);
    - excepciones de dominio y errores de infraestructura con clasificación por severidad.
  - *Modos de operación de logging:*
    - **Modo local-file:** logs en `./logs/statspro.log` con rotación local.
    - **Modo Seq:** levantar Seq con `docker-compose.yml`, enviar logs estructurados al endpoint HTTP de Seq y analizar trazas en UI web.
  - *Rol de `execution_context.py`:*
    - crea el contexto inicial por comando (`correlation_id`, usuario, club activo, comando, timestamp, ambiente);
    - propaga ese contexto hacia casos de uso, servicios y repositorios;
    - serializa metadata homogénea para enriquecer todos los eventos de log.
  - *Criterios de aceptación técnicos:*
    - [ ] AC1: niveles DEBUG/INFO/WARNING/ERROR en formato estructurado JSON.
    - [ ] AC2: `correlation_id` UUIDv4 generado al inicio de cada comando y propagado a use cases/repos.
    - [ ] AC3: redacción de secretos (password, tokens, session-id, hashes sensibles).
    - [ ] AC4: selección de sink por configuración (`local-file` o `seq`) sin cambiar código de negocio.
    - [ ] AC5: en modo Seq, eventos visibles por `correlation_id`, `command_name` y nivel.
  - *Testing mínimo:*
    - Unitario: creación/propagación de `execution_context` y validación de campos obligatorios.
    - Unitario: pruebas de redacción de secretos y normalización de eventos.
    - Integración: flujo CLI completo verificando correlación de logs de punta a punta.
    - Integración opcional observabilidad: prueba de envío de eventos a Seq levantado por `docker-compose`.

## 9.2 Hito 2: Motor de Ingesta y Análisis (v0.2)

**Objetivo del hito:** automatizar la carga desde Ges Deportivo y producir métricas avanzadas confiables.

### Épica H2-E1 — Integración Ges Deportivo

#### ■ US-201: Parseo de Planillas Excel con Pandas

- *Objetivo funcional:* convertir planillas externas en datos persistibles sin transcripción manual.
- *Archivos a crear:* `src/application/use_cases/importar_excel.py`, `src/application/dtos/ingest`, `src/infrastructure/ingest/excel_parser.py`, `src/infrastructure/ingest/ingest_service.py`, `tests/integration/test_excel_import.py`, `tests/fixtures/ges_deportivo_sample.xlsx`.
- *Criterios de aceptación técnicos:*
  - [ ] AC1: validación previa de columnas obligatorias y tipado mínimo esperado.
  - [ ] AC2: comando `preview` sin persistencia con reporte de filas válidas/erróneas.
  - [ ] AC3: merge por DNI para crear o vincular jugador sin duplicados.
  - [ ] AC4: consistencia entre suma de puntos individuales y marcador de partido.
  - [ ] AC5: importación atómica por partido con rollback ante error.
  - [ ] AC6: reporte final con cantidad de creados, actualizados, rechazados y causa.
  - [ ] AC7: filas sin DNI se rechazan explícitamente y quedan registradas en el reporte.
  - [ ] AC8: clubes/competencias faltantes se crean de manera controlada y trazable.
- *Testing mínimo:*
  - Unitario parser: mapeo de columnas, tipado y errores de formato.
  - Unitario ingest-service: estrategias de merge (nuevo/existente) y rechazo de filas inválidas.
  - Integración: importación de fixture real a DB `:memory:` verificando persistencia y rollback.
  - Integración: ejecución de `preview` debe dejar DB sin cambios.

### Épica H2-E2 — Motor Estadístico Pro

#### ■ US-202: Cálculo de Métricas Avanzadas

- *Objetivo funcional:* transformar boxscore crudo en indicadores tácticos accionables.
- *Archivos a crear:* `src/infrastructure/analytics/formulas.py`, `src/application/use_cases/calculo`, `src/application/use_cases/generar_tabla_comparativa.py`, `src/application/dtos/metricas_d`, `tests/unit/test_formulas.py`.
- *Fórmulas mínimas:* eFG %, EFF, PPP, posesiones y porcentaje de rebotes.
- *Criterios de aceptación técnicos:*
  - [ ] AC1: implementación de fórmulas documentadas con salida determinística.
  - [ ] AC2: manejo de división por cero y `NaN/inf` con normalización explícita.
  - [ ] AC3: filtros por temporada, competencia y rango de partidos.
  - [ ] AC4: reporte comparativo Equipo vs Rival por partido.

- [ ] AC5: `formulas.py` libre de acceso a DB (funciones puras sobre DataFrame).
- *Testing mínimo:*
  - Unitario: cobertura completa de fórmulas y casos borde.
  - Unitario: validación de redondeo y formato de salida de DTOs.
  - Integración: use casos analíticos con datos semilla y comparación contra valores esperados.

## Épica H2-E3 — Inteligencia Deportiva (Pandas Engine)

### ■ US-203: Integración del Motor Estadístico

- *Objetivo funcional:* conectar vistas SQL con servicios analíticos sin acoplar fórmula y persistencia.
- *Archivos a crear:* `src/domain/interfaces/analytics_service.py`, `src/infrastructure/analytics/`, `src/application/use_cases/calcular_estadisticas_partido.py`, `tests/integration/test_anal.`
- *Criterios de aceptación técnicos:*
  - [ ] AC1: servicio lee exclusivamente `v_boxscore_completo` y `v_jugador_totales_temporada`.
  - [ ] AC2: contrato `AnalyticsService` desacoplado de detalles de infraestructura.
  - [ ] AC3: columnas estandarizadas y trazables para consumo de UI/reportería.
  - [ ] AC4: errores de lectura/cálculo retornan excepciones de dominio controladas.
- *Testing mínimo:*
  - Unitario con DataFrames en memoria y mocks de repositorio de lectura.
  - Integración end-to-end desde vistas SQL hacia DTO de métricas.
  - Pruebas de regresión de columnas esperadas en vistas consumidas.

## Épica H2-E4 — Gobernanza de Esquema y Migraciones

### ■ US-204 (Nueva): Versionado de Esquema y Migraciones

- *Objetivo funcional:* asegurar evolución de base sin pérdida de datos entre versiones.
- *Archivos a crear:* `src/infrastructure/persistence/sql/migrations/001_init.sql`, `002_*.sql`, `src/infrastructure/persistence/migration_runner.py`, `tests/integration/test_migrations.p`
- *Criterios de aceptación técnicos:*
  - [ ] AC1: tabla `schema_version` creada y mantenida automáticamente.
  - [ ] AC2: runner ejecuta migraciones pendientes en orden y registra resultado.
  - [ ] AC3: soporte de rollback controlado para la última migración aplicada.
  - [ ] AC4: validación al arranque bloquea ejecución ante schema incompatible.
  - [ ] AC5: scripts versionados por release y trazabilidad en changelog técnico.
- *Testing mínimo:*
  - Integración upgrade multi-versión sobre base de datos con datos reales.
  - Integración rollback y verificación de integridad posterior.
  - Integración no-regresión: dataset previo conserva cardinalidad y relaciones.

### 9.3 Hito 3: Visualización Pro y Reporting (v0.3)

**Objetivo del hito:** convertir estadísticas en tableros e informes consumibles para decisiones tácticas.

#### Épica H3-E1 — Dashboards e Informes

##### ■ US-301: Dashboards e Informes Interactivos

- *Objetivo funcional:* exponer líderes y tendencias en CLI/GUI con filtros tácticos.
- *Archivos a crear:* `src/application/use_cases/obtener_lideres_temporada.py`, `src/application/u`  
`src/infrastructure/analytics/chart_generator.py`, `src/infrastructure/reports/cli_leaderb`
- *Criterios de aceptación técnicos:*
  - [ ] AC1: top-5 por rubro (PTS/REB/AST/EFF) en formato tabular legible.
  - [ ] AC2: gráficos de tendencia por jugador/equipo con filtro por temporada y competencia.
  - [ ] AC3: generación de gráfico en tiempo objetivo (<2s) para dataset de referencia (3 temporadas, 20 equipos, 1.200 filas de boxscore).
  - [ ] AC4: consistencia entre valores graficados y valores de vistas SQL.
- *Testing mínimo:*
  - Unitario: ordenamiento de líderes y reglas de desempate.
  - Unitario: transformación DataFrame a serie para gráficos.
  - Integración: reporter CLI + chart generator con datos de base semilla.

##### ■ US-302: Exportación Profesional a PDF

- *Objetivo funcional:* emitir reporte formal de partido/temporada para análisis y difusión interna.
- *Archivos a crear:* `src/application/use_cases/exportar_reporte.py`, `src/application/dtos/repor`  
`src/infrastructure/reports/pdf_generator.py`, `tests/integration/test_pdf_export.py`.
- *Criterios de aceptación técnicos:*
  - [ ] AC1: encabezado con clubes, competencia, fecha y metadata del reporte.
  - [ ] AC2: tablas de boxscore y métricas avanzadas con alineación numérica.
  - [ ] AC3: convención de nombre `boxscore_YYYY-MM-DD_Local_vs_Visitante.pdf`.
  - [ ] AC4: creación automática de `exports/` si no existe.
  - [ ] AC5: manejo de errores de I/O con mensaje de recuperación.
- *Testing mínimo:*
  - Integración: generación de PDF válido a partir de partido semilla.
  - Integración: validación de nombre de archivo y ruta de salida.
  - Integración: caso de error por ruta sin permisos.

##### ■ US-303 (Nueva): Scouting de Rival Pre-Partido

- *Objetivo funcional:* entregar reporte táctico del rival previo al encuentro.

- *Archivos a crear:* `src/infrastructure/persistence/sql/views_scouting.sql`, `src/application/usage_scouting.py`, `src/application/dtos/scouting_dto.py`, `tests/integration/test_scouting_rival.py`.
- *Criterios de aceptación técnicos:*
  - [ ] AC1: comparación de últimos N partidos configurable por parámetro.
  - [ ] AC2: detección de patrones (rachas, tendencia de tiro, pérdidas recurrentes).
  - [ ] AC3: filtros por competencia/categoría con resultados consistentes.
  - [ ] AC4: reporte exportable/visible con conclusiones y datos fuente.
- *Testing mínimo:*
  - Unitario: agregaciones históricas y funciones de detección de patrones.
  - Integración: vistas de scouting con dataset histórico de ejemplo.
  - Integración: consistencia de filtros por competencia y ventana N.

## 9.4 Hito 4: Interfaz Multiplataforma y Entrega (v1.0)

**Objetivo del hito:** migrar a experiencia visual completa y cerrar hardening de release.

### Épica H4-E1 — Interfaz de Usuario Adaptable (GUI)

#### ■ US-401: Implementación de Interfaz Flet (Desktop/Mobile)

- *Objetivo funcional:* ofrecer una interfaz visual completa reutilizando casos de uso ya consolidados.
- *Archivos a crear:* `src/infrastructure/ui/flet/app.py`, `src/infrastructure/ui/flet/screens/dashboard.py`, `src/infrastructure/ui/flet/screens/game_entry_screen.py`, `src/infrastructure/ui/flet/screens/player_profile_screen.py`, `src/infrastructure/ui/flet/components/chart.py`.
- *Criterios de aceptación técnicos:*
  - [ ] AC1: arranque inicial en menos de 3 segundos en entorno objetivo (desktop con 4GB RAM, SSD y Python 3.11+).
  - [ ] AC2: navegación estable entre pantallas principales sin pérdida de estado.
  - [ ] AC3: soporte tema claro/oscuro y diseño responsive desktop/mobile.
  - [ ] AC4: formularios con validación visual y mensajes de error entendibles.
- *Testing mínimo:*
  - Unitario: validadores de formularios y mapeo de estado de pantalla.
  - Integración UI: flujo navegación Dashboard → GameEntry → PlayerProfile.
  - Prueba manual guiada en al menos dos plataformas (desktop + mobile).

#### ■ US-402: Backup y Estabilización Final

- *Objetivo funcional:* proteger continuidad operativa mediante respaldo/restauración y hardening final.
- *Archivos a crear:* `src/application/use_cases/exportar_backup.py`, `src/application/use_cases/restaurar.py`, `src/infrastructure/persistence/backup_service.py`, `tests/integration/test_backup_restore.py`.
- *Criterios de aceptación técnicos:*
  - [ ] AC1: exportación de backup completa con metadata de versión y fecha.
  - [ ] AC2: restauración íntegra sobre instancia limpia y sobre instancia con datos.

- [ ] AC3: verificación automática post-restore (conteo de tablas críticas y checksum lógico).
- [ ] AC4: checklist de estabilización ejecutada (rendimiento, errores críticos, regresión).
- *Testing mínimo:*
  - Integración: ciclo exportar-restaurant con comparación de datos clave.
  - Integración: restauración desde backup de versión anterior con migraciones aplicadas.
  - Regresión: ejecución de casos críticos tras restore (auth, carga partido, reportes).
- **US-403 (Nueva): Seguridad de Datos Local**
  - *Objetivo funcional:* endurecer credenciales, sesión y almacenamiento local de datos.
  - *Archivos a crear:* `src/infrastructure/security/credential_policy.py`, `src/infrastructure/security/db_encryption_adapter.py`, `docs/security/password_compromised_tests/unit/test_credential_policy.py`.
  - *Criterios de aceptación técnicos:*
    - [ ] AC1: credenciales de mínimo 12 caracteres y validación de fortaleza.
    - [ ] AC2: bloqueo efectivo de contraseñas comprometidas mediante lista local versionada.
    - [ ] AC3: actualización mensual de la lista de comprometidas siguiendo `docs/security/password_compromised`.
    - [ ] AC4: validación en CI de versión vigente y checksum de la lista de comprometidas.
    - [ ] AC5: hashing robusto (`bcrypt/Argon2`) con salt por usuario.
    - [ ] AC6: cifrado opcional de base local con estrategia compatible SQLCipher.
    - [ ] AC7: política de sesión segura (expiración, revocación y limpieza idempotente).
    - [ ] AC8: trazabilidad de eventos de seguridad sin exponer datos sensibles.
  - *Testing mínimo:*
    - Unitario: política de contraseñas y bloqueo por comprometidas.
    - Unitario: ciclo hash/verificación y migración de hashes legados.
    - Integración: expiración/revocación de sesión y limpieza de credenciales en logout.
    - Integración: acceso a DB cifrada/no cifrada según configuración.

## 10 Reglas de Negocio Consolidadas

---

**Objetivo:** centralizar reglas obligatorias del dominio para que el desarrollo y testing sean consistentes en todas las historias de usuario.

### Identidad y seguridad

- Email de usuario único por sistema.
- Contraseña nunca persistida en texto plano ni en logs.
- Sesión requiere usuario autenticado y, para comandos operativos, club activo.

### Jugadores, clubes y afiliaciones

- DNI de jugador único cuando está informado.

- Un jugador no puede tener dos vínculos activos superpuestos con el mismo club.
- Historial de afiliación debe respetar coherencia temporal (`fecha_hasta >= fecha_desde`).

### Competencias, inscripciones y listas

- Una inscripción es única por club + competencia + categoría + temporada.
- Cada inscripción tiene una única lista de buena fe asociada.
- Solo jugadores habilitados en lista pueden figurar en carga oficial de partido.

### Partidos y estadísticas

- Un partido no puede tener el mismo club como local y visitante.
- Toda carga de partido y boxscore es atómica (todo o nada).
- En estadísticas de tiro: convertidos  $\leq$  lanzados y todos los valores son no negativos.
- Los puntos por jugador deben ser consistentes con T1C, T2C y T3C.
- Minutos por jugador no pueden exceder el máximo reglamentario definido por competencia.

### Analítica y reporting

- Fórmulas avanzadas deben manejar división por cero y no devolver NaN/inf.
- Reportes y dashboards se construyen sobre vistas SQL normalizadas y versionadas.
- Toda exportación (PDF/backup) debe ser trazable en logs con timestamp y resultado.

## 11 Requisitos No Funcionales (NFR)

| ID    | Requisito           | Medición / Umbral                                                          | Severidad  |
|-------|---------------------|----------------------------------------------------------------------------|------------|
| NFR-1 | Portabilidad        | Ejecución nativa en Windows 10+, Android 9+, iOS 14+.                      | Bloqueante |
| NFR-2 | Rendimiento Ingesta | Procesamiento de Excel con Pandas <5 seg.                                  | Alta       |
| NFR-3 | Fiabilidad Datos    | Integridad referencial en SQLite (Foreign Keys activas).                   | Bloqueante |
| NFR-4 | Usabilidad          | Carga de un partido completo en <3 clics (desde la selección del archivo). | Media      |

## 12 Definición de "Hecho"(Definition of Done)

Una historia de usuario se considera **Hecha** cuando:

1. El código está integrado en la rama principal sin conflictos.
2. Pasa pruebas unitarias e integración con cobertura >80 % en componentes no críticos.



3. Los componentes críticos alcanzan cobertura >95 % (en esta versión: autenticación, persistencia transaccional, rollback y fórmulas estadísticas).
4. La clasificación crítica/no crítica se define y versiona en el catálogo técnico del backlog.
5. Se respeta la arquitectura de capas (sin imports cruzados entre `domain` y `infrastructure`).
6. Si hay cambios SQL, la vista/tabla está versionada y cubierta por test de integración con datos semilla.
7. Si la US persiste múltiples tablas, existe evidencia de transacción atómica y rollback probado.
8. El ADR correspondiente fue aprobado y documentado.
9. La interfaz fue validada en al menos dos plataformas (ej. Desktop y Mobile).
10. Métodos públicos relevantes incluyen docstring y trazabilidad mínima en logs de operación.

## 12.1 Catálogo técnico de criticidad

El catálogo de criticidad es un artefacto vivo del backlog de arquitectura donde se lista cada módulo, su nivel de criticidad (crítico/no crítico), justificación, dueño técnico y evidencia de cobertura objetivo. Su mantenimiento se realiza en cada refinamiento de hito y es prerequisite para validar el DoD.

**Formato y ubicación:** archivo versionado `docs/catalogo-criticidad.md` con tabla mínima: Módulo | Criticidad | Justificación | Owner | Cobertura objetivo | Evidencia. El catálogo se inicializa en Hito 1 y se actualiza en cada planificación de sprint. **Criterios de clasificación:** un módulo se marca como crítico si cumple al menos uno de estos puntos: (1) controla acceso/autenticación/autorización; (2) persiste datos en múltiples tablas con transacciones; (3) impacta integridad histórica del dato deportivo; (4) implementa cálculos estadísticos oficiales usados en decisiones tácticas. **Gobernanza:** owner primario Arquitecto Técnico; co-owner QA Lead. Revisión obligatoria en kickoff de hito y en cada planificación de sprint.

## 13 Hoja de Ruta y Futuras Expansiones

Para transformar StatsPro Basketball en una plataforma líder de inteligencia deportiva, se proponen las siguientes fases de expansión post-v1.0:

### 13.1 Hito 5: Ecosistema Conectado y Colaborativo (v1.5)

**Objetivo:** Romper el aislamiento local para permitir la colaboración entre cuerpos técnicos.

- **Sincronización Cloud Híbrida:** Implementación de un backend (ej. FastAPI + PostgreSQL) para respaldar datos en la nube y permitir el uso multidispositivo (PC en oficina, Tablet en cancha).
- **Módulo de Exportación para Redes:** Generación de "Match Cards" automáticas con las figuras del partido para compartir directamente en Instagram/X del club.
- **Base de Datos Compartida de Scouting:** Repositorio comunitario donde DTs pueden compartir datos de rivales (bajo consentimiento), creando un mapa de scouting regional.

### 13.2 Hito 6: Inteligencia Táctica Basada en IA (v2.0)

**Objetivo:** Pasar del análisis descriptivo (¿qué pasó?) al análisis prescriptivo (¿qué debería pasar?).

- **Motor de Recomendaciones Tácticas:** Uso de modelos de ML para sugerir quintetos iniciales basados en el "Match-up" contra el rival y el rendimiento reciente.
- **Detección Automática de Patrones:** Identificación de rachas de tiro o debilidades defensivas durante la carga de datos, alertando al DT sobre tendencias críticas.
- **Integración de Video Scouting:** Vinculación de eventos estadísticos con clips de video (ej. vía integración con YouTube o archivos locales) para sesiones de video-análisis rápidas.

### 13.3 Hito 7: Live Stats y Scouting en Tiempo Real (v3.0)

**Objetivo:** Capturar datos en el momento en que ocurren, eliminando la dependencia exclusiva de Excel externos.

- **Módulo de Captura en Vivo:** Interfaz táctil optimizada para que un asistente cargue eventos (tiros, pérdidas, faltas) en tiempo real desde una tablet.
- **Live Dashboard para el Banco:** Visualización de métricas avanzadas (PPP, eFG
- **Control de Carga y Fatiga:** Integración con wearables o entrada manual de minutos para gestionar el esfuerzo físico de los jugadores jóvenes a lo largo de torneos intensos.